



Universidad Nacional Experimental de Guayana
Vicerrectorado Académico
Proyecto de Carrera: Ingeniería en Informática
Unidad Curricular: Lenguajes y Compiladores

ANÁLISIS SINTÁCTICO

LOS ASTRONAUTAS

“EXPLORANDO EL UNIVERSO DE LOS LENGUAJES, UN NODO A LA VEZ”

Profesor:
Félix Márquez

Presentado por:
C.I.: V-25.933.680. Alburquerque Sheen
C.I.: V-30.907.427. Antoima Mariangel
C.I.: V-28.475.271. García Carlos
C.I.: V-24.848.424. Varguillas Génesis



ACTIVIDAD 1

ÁRBOL DE SINTAXIS ABSTRACTA (AST)

POR: MARIANGEL ANTOIMA

ÁRBOL DE SINTAXIS ABSTRACTA (AST)

Es una representación intermedia del código que elimina detalles sintácticos, conservando solo la estructura esencial del programa.

CARACTERÍSTICAS:

- Abstracción sintáctica
- Jerarquía semántica
- Facilidad de procesamiento
- Independencia de la sintaxis



EJEMPLOS:

=====

ÁRBOL DE SINTAXIS ABSTRACTA - EJEMPLO 1

=====

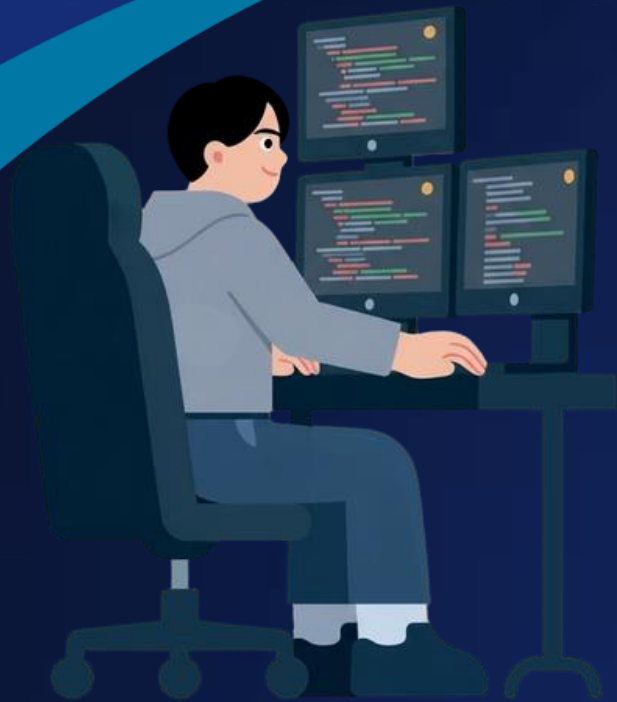
```
├ operador_binario: * (línea 1)
  │
  └ operador_binario: + (línea 1)
    │
    ├── numero: 5 (línea 1)
    │
    └ numero: 3 (línea 1)
      │
      └ numero: 2 (línea 1)
```

=====

ÁRBOL DE SINTAXIS ABSTRACTA - EJEMPLO 2

=====

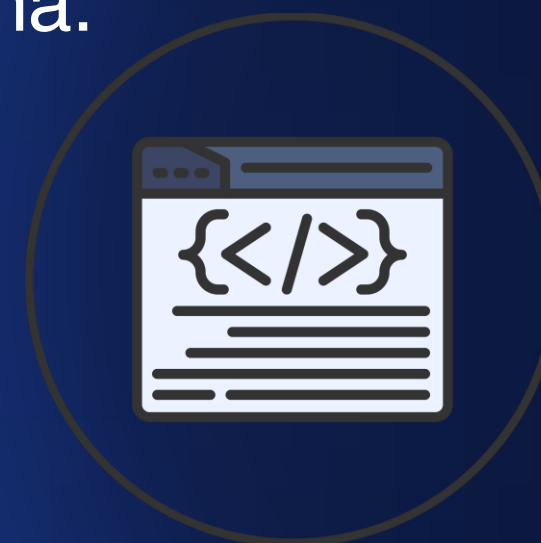
```
├ if (línea 1)
  │
  └ operador_comparacion: > (línea 1)
    │
    ├── variable: x (línea 1)
    │
    └ numero: 0 (línea 1)
      │
      └ asignacion: = (línea 1)
        │
        ├── variable: y (línea 1)
        │
        └ numero: 1 (línea 1)
```

IMPLEMENTACIÓN DEL AST

La implementación del AST se realizó en Python mediante la clase `NodoAST`, que almacena:

- Tipo
- Valor
- Hijos
- Ubicación (errores)



EJEMPLOS PROBADOS:

- Expresión aritmética
- Sentencia `if`
- Bucle `while`

RESULTADOS:

Ejemplo	Código Fuente	Estado	Observación
1	<code>(5 + 3) * 2</code>	Correcto	AST generado con operador <code>*</code> como raíz
2	<code>if (x > 0) { y = 1; }</code>	Correcto	AST generado con nodo <code>if</code> y sus hijos
3	<code>while (i < 10) { i = i + 1; }</code>	Correcto	AST generado con nodo <code>while</code> y sus hijos



ACTIVIDAD 2

COMPARACIÓN LL VS LR

POR: MARIANGEL ANTOIMA

COMPARACIÓN DE ESTRATEGIAS LL Y LR

LL (descendente):

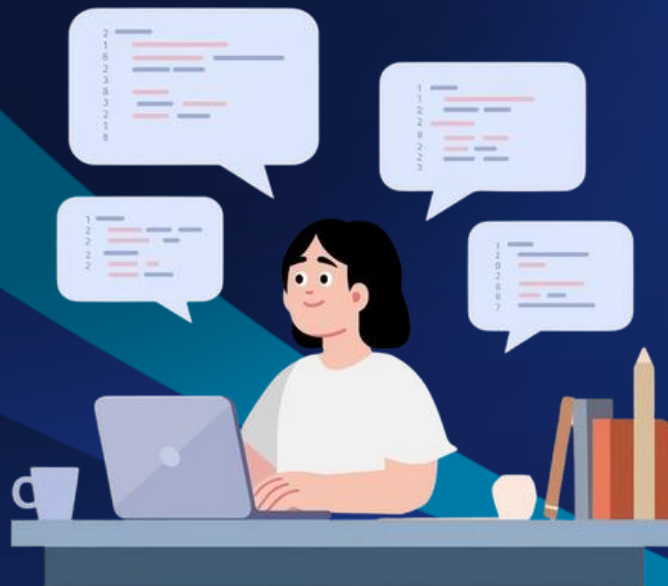
- Parte del símbolo inicial
- Expande no terminales

LR (ascendente):

- Parte de los tokens
- Reduce a no terminales

Gramáticas:

- LL: sin recursión por la izquierda
- LR: producción para listas de identificadores



RESULTADOS DE LAS PRUEBAS

Parser	Pruebas Realizadas	Pruebas Exitosas	Tasa de Éxito
Parser LL (Recursivo Descendente)	9	9	100%
Parser LR	10	10	100%



Ambas estrategias funcionan correctamente. LL es más sencillo de implementar manualmente, mientras que LR es más potente para gramáticas complejas.



ACTIVIDAD 3

GENERADORES DE ANALIZADORES SINTÁCTICO

POR: SHEEN ALBURQUERQUE

GENERADORES DE ANALIZADORES SINTÁCTICOS (METACOMPILADORES)

Herramientas que, a partir de una especificación formal de una gramática(BNF/EBNF), generan automáticamente el código fuente de un parser.

Generador	Tipo de Parser	
ANTLR	LL(*)	
Bison	LALR(1)	
JavaCC	LL(k)	

GESTIÓN DE ERRORES

Un parser efectivo debe:

1. Detectar errores
2. Reportarlos con precisión (línea y columna)
3. Recuperarse para continuar el análisis

Estrategias de Recuperación:

Estrategia	Descripción
Modo Pánico	Descarta <i>tokens</i> hasta encontrar un sincronizador (ej: ;)
Inserción de Tokens	Inserta <i>tokens</i> faltantes para corregir el error
Recuperación a Nivel de Frase	Correcciones locales en el punto del error

Ejemplo de reporte:

❓ ERROR [Línea 1, Columna 9]: Se esperaba ',' o ';', se encontró 'id'

RESULTADOS DE LA IMPLEMENTACIÓN

1. Reporte de Errores con Ubicación:

❓ Error en línea 1, columna 9: Se esperaba ',' o ';', se encontró 'id'

2. Modo Pánico (Panic Mode):

❓ Entrada: id, id, error, error2, ;
Tokens descartados: error, error2
Recuperado en ;

3. Sugerencias de Corrección (Confianza):

Token Incorrecto	Token Correcto	Ratio de Con
pront	print	0.8
prnt	print	0.89
whil	while	0.89
retrn	return	0.91

CONCLUSIÓN:

ACTIVIDADES 1, 2 Y 3

Actividad 1: AST- Representación jerárquica del código que elimina detalles superfluos- Permite que las fases posteriores (semántica, optimización) trabajen eficientemente

Actividad 2: LL vs LR- Ambas estrategias alcanzaron el 100% de éxito en las pruebas- LL: más sencillo, ideal para gramáticas simples- LR: más potente, adecuado para gramáticas complejas

Actividad 3: Generadores y Errores- Herramientas como ANTLR, Bison y SLY automatizan la construcción de parsers- Técnicas de recuperación: modo pánico, inserción de tokens y sugerencias con confianza

Actividad 4: experimento de carga, subconjunto de YAML

Parsers: C vs C# vs Python

Carlos David Garcia



HERRAMIENTAS

Criterio	LL	
Complejidad	Baja	Alta
Flexibilidad	Gramáticas restrictivas	Grande
Dirección	Descendente	Ascendente



LL (ANTLR):*
Raíz
↓
Expansión
↓
Lookahead
↓
Tokens (descendente)



LALR (Bison):
Raíz (ascendente)
↑
Reducción
↑
[Stack + Tabla]
↑
Tokens

Subconjunto de YAML

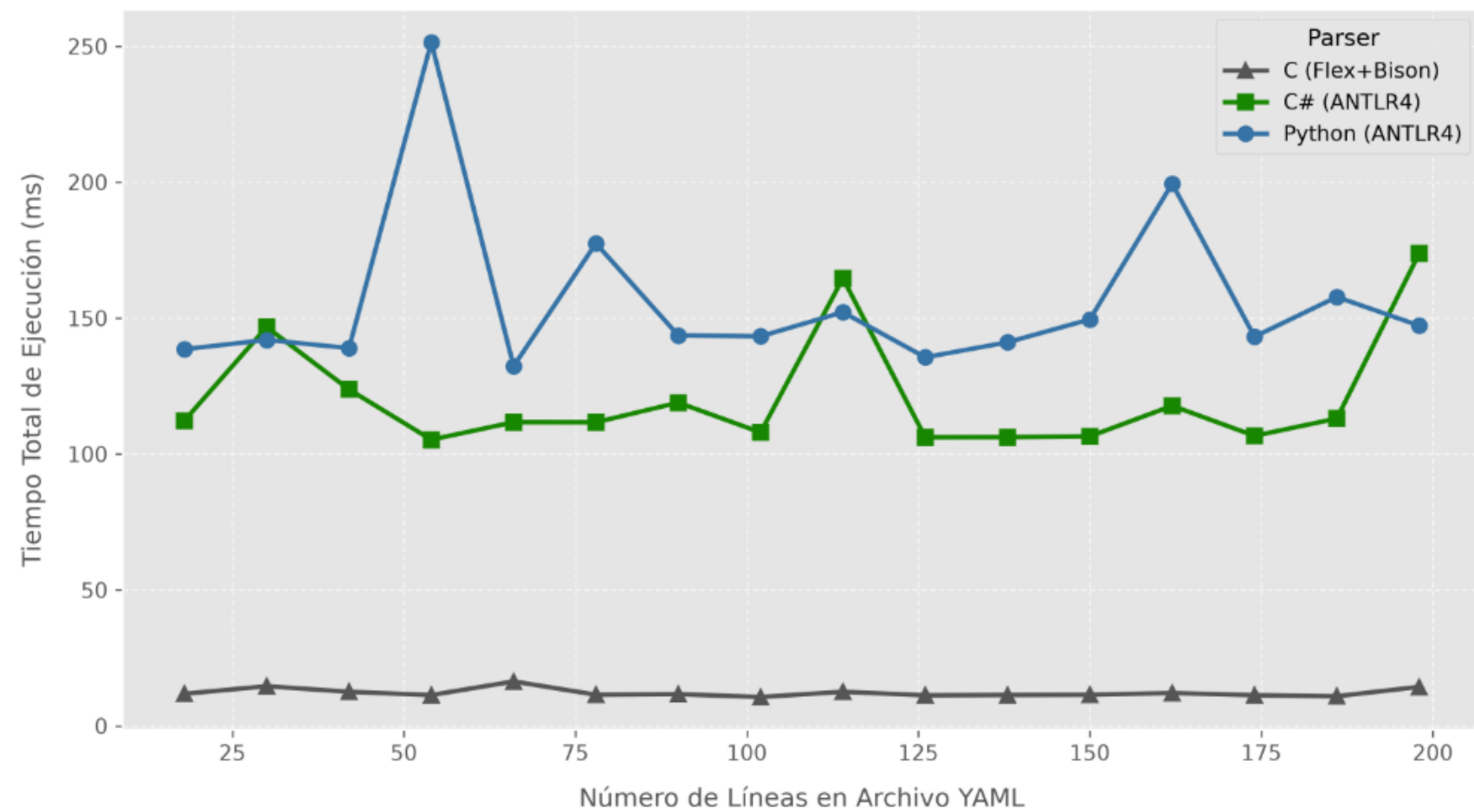
- Nivel de indentación estricto de 2 espacios
- Comentarios no permitidos para facilidad de implementación
- version: Especificación de versión (ej. '3.8').
- services: Definición de servicios (ej. web), propiedades de image y referencias a networks.
- networks: Definición de redes de contenedores con sus atributos driver e ipam -> config -> subnet.

Ejemplo YAML

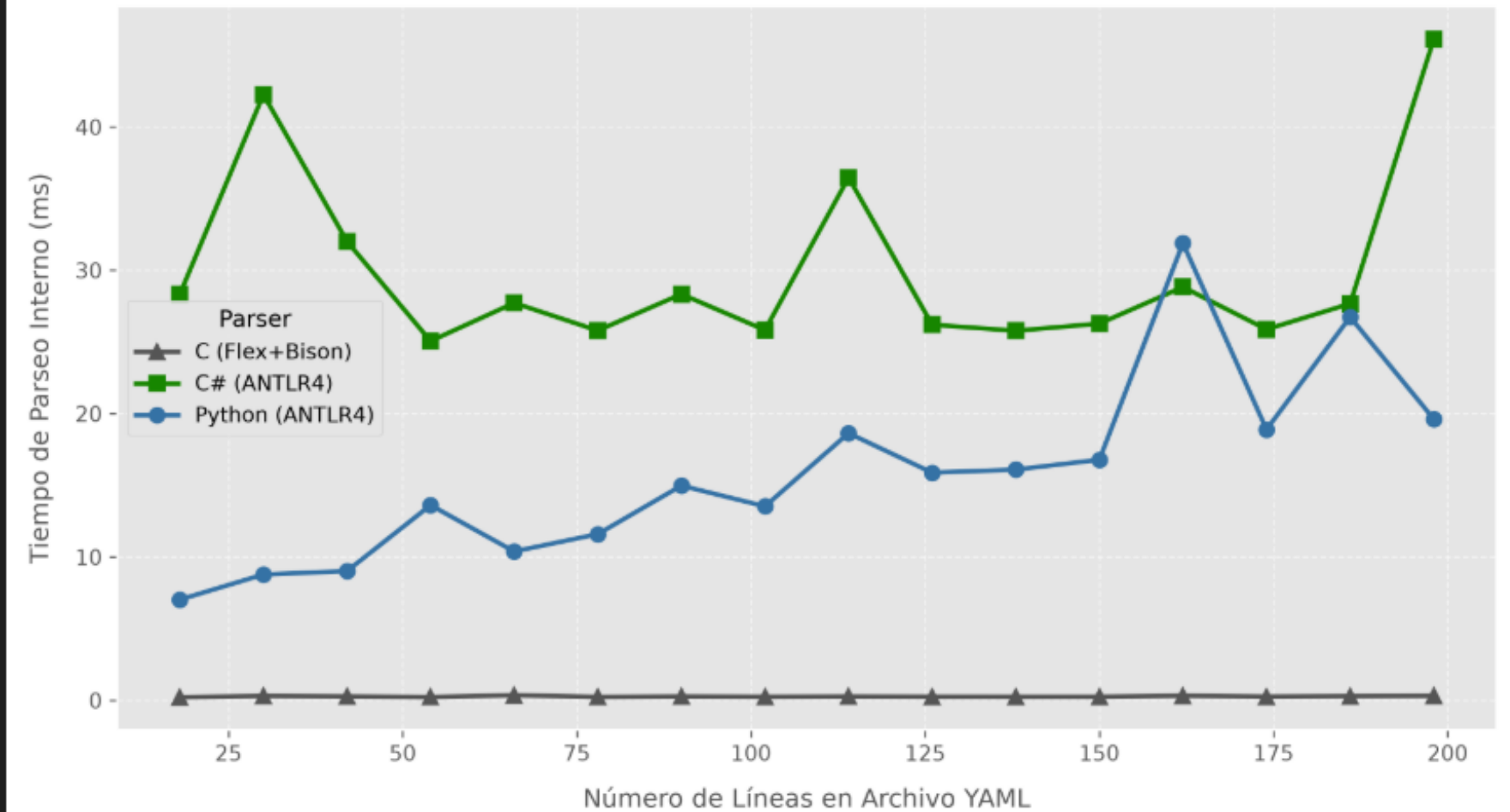
```
version: '3.8'
Run All Services
services:
  Run Service
  web:
    image: nginx:latest
    networks:
      - net_1
      - net_2
networks:
  net_1:
    driver: bridge
    ipam:
      config:
        subnet: 172.20.1.0/24
  net_2:
    driver: bridge
    ipam:
      config:
        subnet: 172.20.2.0/24
```

Resultados

Comparativa de Tiempo Total de Ejecución (Proceso Completo)



Comparativa de Tiempo Interno de Parseo (Parse Time)



La diferencia entre ellos

Criterio / Aspecto	C con
Algoritmo de parsing	Autómata c sola pasada
...	Bajo (sin es

Conclusiones

- El parser y el lenguaje determinan en gran medida el rendimiento
- C para parsear muchos archivos pequeños
- Lenguajes compilados suelen ser mas rapidos que los interpretados
- La identacion requiere cuidado para trabajarla de manera correcta



ACTIVIDAD 5

ASISTENTE DE PROGRAMACIÓN HÍBRIDO

POR: GÉNESIS VARGUILLAS

Compiladores Tradicionales



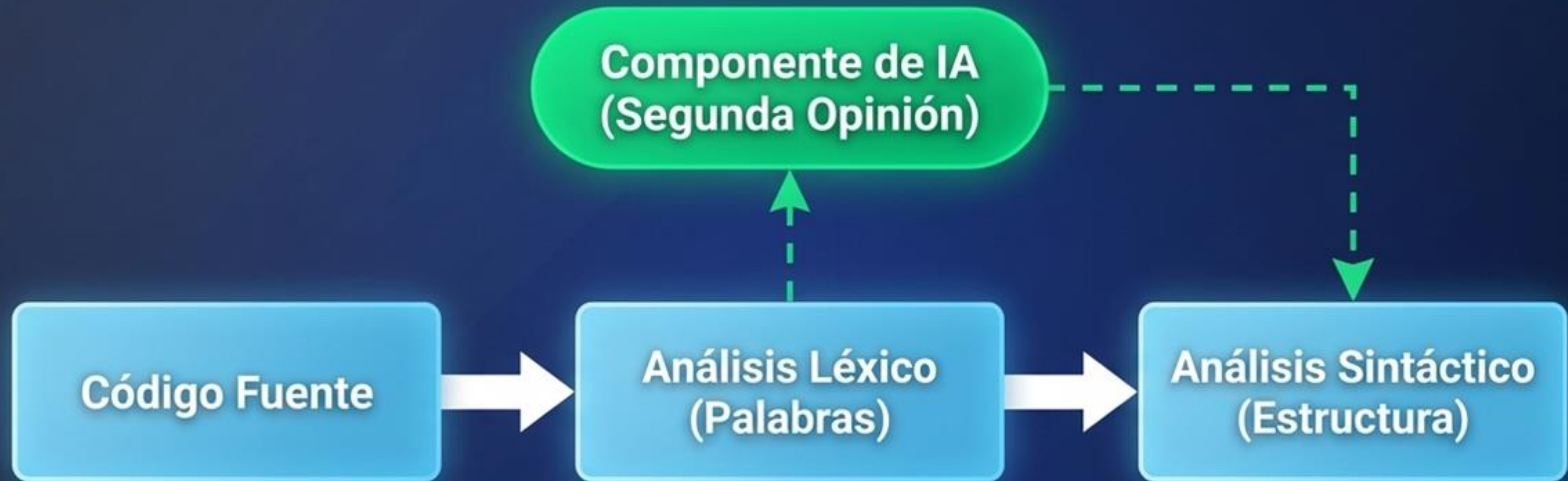
Estrictos por naturaleza. Ante cualquier elemento desconocido o error mínimo, el proceso se detiene y reporta un fallo fatal.

La Filosofía UnegScript



Tolerancia activa. Sistema de dos capas: el motor tradicional trabaja junto a **una** IA que actúa como **red de seguridad** ante la ambigüedad, sugiriendo correcciones sin detener el flujo.

ARQUITECTURA



Si el sistema encuentra algo ambiguo, no se detiene.
Consulta a la IA y, si la confianza es alta, corrige y continúa.

prontoprint ✓



Reconocimiento del Código.

Identificación automatizada de palabras clave y símbolos. Ante cuasi-coincidencias (errores de tipeo), el sistema evalúa, detecta la anomalía y aplica una corrección automática instantánea.



Comprensión de la Estructura.

Reconocimiento de reglas gramaticales. Si falta un elemento estructural, el análisis no se corta. La IA infiere el componente ausente, genera la sugerencia, y el sistema continúa construyendo el árbol del programa.



ejemplo_prueba.us

```
x = 10
if x > 5:
    pront("X is greater than 5")
else:
    print("X is not greater than 5")
```

Escenario de prueba real: código con fallos léxicos y sintácticos intencionales.

Salida Principal

```
x = 10
if x > 5:
    print("X is
greater than 5")
else:
    print("X is not
greater than 5")
```

Log de Sugerencias

[ALTA CONFIANZA]

'pront' -> 'print'

[ESTRUCTURA] Se
sugiere agregar ':' en
línea 4



Escalabilidad Confirmada.

El tiempo de análisis crece de forma estrictamente proporcional al tamaño del programa.
La integración de IA no genera cuellos de botella ni comportamientos anómalos.

Comprensión
Estructural

25%

Reconocimiento
Léxico + Consultas
IA

75%

Desglose de Rendimiento.

La mayor carga de procesamiento se concentra en la primera etapa (análisis léxico), ya que es ahí donde ocurren la mayoría de las resoluciones de ambigüedad y consultas al motor de Inteligencia Artificial.

CONCLUSIONES



Futuro: Modelos
de Lenguaje (LLMs)

Lo mejor de dos mundos: un sistema que aprende a reconocer y corregir errores sin intervención humana. Próxima evolución: reemplazar la lógica de comparación con actual con LLMs avanzados manteniendo la arquitectura base.



**¡MUCHAS
GRACIAS!**